INFOTACT SOLUTIONS

OMNISPACE

Spatial UI System

Week 2 – Gesture Mapping Report

ROLE
UX Research & Spatial Architecture

RESPONSIBILITY
Member 1

FOCUS
Column 1 — Gesture Mapping: Gaze Hover, Pinch, and Pinch-and-Drag

REVIEW USE
Internship mentor review

Report scope
This report focuses only on interaction architecture and UX research for Week 2. It does not claim that an OmniSpace prototype has been implemented or user-tested.

Contents

1. Introduction and Objectives ····· 3
2. Gaze Hover Interaction ····· 4
3. Pinch Interaction ····· 5
4. Pinch-and-Drag Interaction ····· 6
5. Gesture Mapping Table ····· 7
6. Interaction Architecture ····· 8
7. Gesture Interaction Flow Diagram ····· 8
8. Error Handling and User Feedback ····· 9
9. UX Recommendations ····· 9
10. Conclusion ····· 10
11. References ····· 11

How to read this report

The report uses a simple interaction pattern: gaze identifies the target, pinch commits an action, and pinch-and-drag supports continuous movement. Established platform guidance is cited as evidence. OmniSpace-specific behaviors are presented as design recommendations for future validation, not as tested findings.

Evidence labels

Established platform guidance
behavior documented by Apple or Microsoft for spatial-computing systems.

Design recommendation
a proposed OmniSpace behavior that should be prototyped and evaluated later.

Open validation point
a detail that requires user testing or technical feasibility review.

1. Introduction

OmniSpace is a proposed spatial user interface system in which digital windows, menus, and tools can appear around the user instead of being confined to a flat screen. In this kind of interface, the user needs a clear way to express attention, selection, and movement without relying on a mouse or physical touch surface. Gesture mapping converts those intentions into consistent system behaviors.

Gesture-based interaction matters because spatial interfaces have more freedom and more ambiguity than a 2D screen. A person may look at an item without wanting to select it. A hand may be visible but not intentionally acting. The interaction model therefore needs a separation between targeting and committing. Microsoft describes “gaze and commit” as a far-input model in which gaze indicates the target and a secondary action completes the interaction.[3] Apple similarly instructs users to look at an element and tap the thumb and index finger together to select it.[1]

For this Week 2 task, the selected interaction language is deliberately small: Gaze Hover, Pinch, and Pinch-and-Drag. A limited gesture vocabulary can make the system easier to learn, easier to explain to a mentor, and easier to validate later. The aim is not to design every gesture, but to define the architecture and feedback rules for three high-value actions.

2. Objectives

- Map the relationship between user intent, tracked input, system state, and visual feedback.
- Define what happens before, during, and after Gaze Hover, Pinch, and Pinch-and-Drag.
- Reduce accidental activation by separating “looking at” from “committing to” an action.
- Specify feedback that lets users understand what the system thinks they are targeting or holding.
- Identify likely errors and provide recovery behaviors that do not punish the user.
- Create a foundation that can be prototyped and user-tested later without claiming that testing has already happened.

Working principle
Target first. Commit second. Manipulate continuously only while the system clearly shows that an object is being held.

3. Gaze Hover Interaction

What is Gaze Hover?

Gaze Hover is a non-committing focus state. When the user looks toward a floating menu item or window control, OmniSpace treats that item as the current target and provides a subtle highlight. The user has not selected anything yet. This distinction is important because eye gaze is fast but can also move naturally across many objects.

How does it work?

A spatial system can intersect a gaze ray with virtual content to estimate which object is being targeted. Microsoft explains that head-gaze or eye-gaze can be used to determine a user's direction and that visual or auditory indication can build confidence about the target.[3] For OmniSpace, the hover state should be stable enough to recognize but light enough that it does not feel like the interface is reacting to every eye movement.

Step-by-step interaction flow

1. The user looks toward a floating menu, window, or control.
2. The system identifies the nearest eligible interactive target along the gaze direction.
3. The target enters a hover state after a short stability threshold; the exact duration is a future validation point.
4. The system highlights the target and, where useful, shows a small label or focus ring.
5. If the user looks away, the target returns to its default state without committing an action.

Real-world example

The user looks at a "Calendar" icon in a floating OmniSpace menu. The icon gains a soft outline and a slightly brighter surface. Nothing opens yet. The user can look at "Mail" instead, and the highlight follows the new target.

Expected visual feedback

- Subtle outline, glow, or surface contrast around the focused item.
- Optional small focus marker for head-gaze fallback; avoid a distracting cursor for eye-gaze unless needed.[3]
- A clear return to the default state when gaze leaves the target.
- No modal change, navigation, or destructive action during hover alone.

Possible errors and solutions

- Unstable gaze: add a short stability threshold and keep the highlight forgiving.
- Small target: enlarge the interactive hit area and provide a head-gaze or controller fallback.
- Accidental focus: do not open or activate on gaze alone.
- User cannot tell what is focused: strengthen the highlight slightly or add a brief label.

4. Pinch Interaction

What is Pinch?

Pinch is the primary commit gesture in this report: the user brings the thumb and index finger together once to select the currently focused target. Apple describes this gesture as a way to select an app, open a link, and activate interface elements.[1] In OmniSpace, Pinch should behave like a click only when a valid target is already focused.

How does it work?

The interaction combines a spatial target from gaze with a discrete hand action. This reduces the need for the user to point with the hand and helps the system distinguish attention from activation. Research on Gaze + Pinch similarly describes gaze as selecting targets while indirect freehand gestures manipulate the object being looked at.[4]

Step-by-step interaction flow

1. The user looks at an eligible target until the hover state is visible.
2. The user brings thumb and index finger together in a short pinch.
3. The system checks that the target is still valid and that the pinch is intentional.
4. The system commits the action: select, open, toggle, or begin a hold state depending on context.
5. The target provides immediate confirmation, such as a pressed state, opening transition, or selection mark.

Real-world example

The user looks at a floating “Notes” tile. The tile highlights. The user pinches once. OmniSpace opens the Notes window at the tile’s location and returns focus to the new window.

Expected visual feedback

- Hover highlight remains visible until the pinch is recognized.
- A short pressed or ripple state confirms the commit.
- The selected object changes state immediately, for example by opening or showing a checkmark.
- If the pinch is not recognized, the system keeps the hover state and does not pretend that the action succeeded.

Possible errors and solutions

- Missed pinch: show a short recognition cue and allow a second attempt without moving the target.
- Accidental pinch: require gaze focus plus a valid pinch transition.
- Hand not visible: display a neutral prompt such as “Move your hand into view.”
- Repeated activation: debounce rapid pinches and make toggles visibly stateful.

5. Pinch-and-Drag Interaction

What is Pinch-and-Drag?

Pinch-and-Drag is a continuous manipulation gesture. The user looks at a draggable object or its window bar, pinches and holds to grab it, moves the hand while maintaining the pinch, and releases to place it. Apple documents the same basic pattern for moving windows, content, and objects: look at the item, pinch and hold, drag, then let go.[1]

How does it work?

The key design requirement is state clarity. The system should make the transition from “focused” to “grabbed” obvious. During movement, the object should follow the hand in a stable and predictable way. The release should be forgiving and should not cause the window to jump unexpectedly.

Step-by-step interaction flow

1. The user looks at a draggable window bar or object.
2. The target enters the hover state and exposes a grab affordance, such as a handle highlight.
3. The user pinches and holds. The system changes the target to a grabbed state.
4. The user moves the hand while maintaining the pinch; the window follows with an appropriate spatial offset.
5. The user releases the pinch. The window settles in the new position and the grabbed feedback ends.

Real-world example

The user opens a project dashboard and wants it beside a reference window. They look at the dashboard’s window bar, pinch and hold, move it to the right, and release. The dashboard remains open in the new position.

Expected visual feedback

- Grabbed state: stronger outline, handle glow, or slight elevation cue.
- Continuous movement: the window follows without lag that makes the user unsure whether it is held.
- Release: a settling animation or brief placement confirmation.
- Boundary feedback when the window reaches a safe placement region or cannot move further.

Possible errors and solutions

- Grab starts on the wrong surface: limit dragging to an explicit window bar or handle.
- Pinch breaks during movement: keep the object selected briefly and allow re-grab without reset.
- Object occludes the hand: preserve a visible outline or ghost position.
- Window moves too far or becomes unreachable: apply spatial bounds and provide a recenter action.

6. Gesture Mapping Table

The table below defines the first interaction architecture for OmniSpace. The "system response" column is the intended behavior for the proposed design; it is not a report of implemented functionality.

Gesture	User action	System response	Visual feedback	Purpose	Possible error
Gaze Hover	Look at an interactive item.	Identify target; enter non-committing focus state.	Subtle highlight or focus ring; no navigation.	Targeting and orientation.	Gaze instability, small target, unclear focus.
Pinch	Bring thumb and index finger together once while target is focused.	Commit select/open/toggle action.	Pressed state, ripple, open transition, or selection mark.	Discrete selection.	Missed pinch, accidental activation, hand out of view.
Pinch-and-Drag	Pinch and hold a draggable target, move, then release.	Grab, continuously reposition, then place.	Grab outline, movement tracking, release/settle cue.	Spatial manipulation.	Wrong surface, broken pinch, unreachable placement.

Mapping rules

- Gaze Hover never commits by itself. It only establishes focus.
- Pinch is the default commit action for focused controls.
- Pinch-and-Drag requires a draggable target and a visible grabbed state.
- Every gesture has a recoverable failure path: the system should explain or expose the next attempt rather than silently changing context.
- Feedback is part of the interaction contract, not decoration.

Design boundary
This table is a design specification for future prototyping. It does not report measured accuracy, completion time, comfort, or user satisfaction.

7. Interaction Architecture

The three gestures work as a layered architecture rather than as unrelated shortcuts. The gaze layer answers "Where is the user's attention directed?" The pinch layer answers "Does the user want to commit?" The pinch-and-drag layer answers "Is the user holding this object and moving it?" This separation reduces ambiguity and makes the behavior explainable.

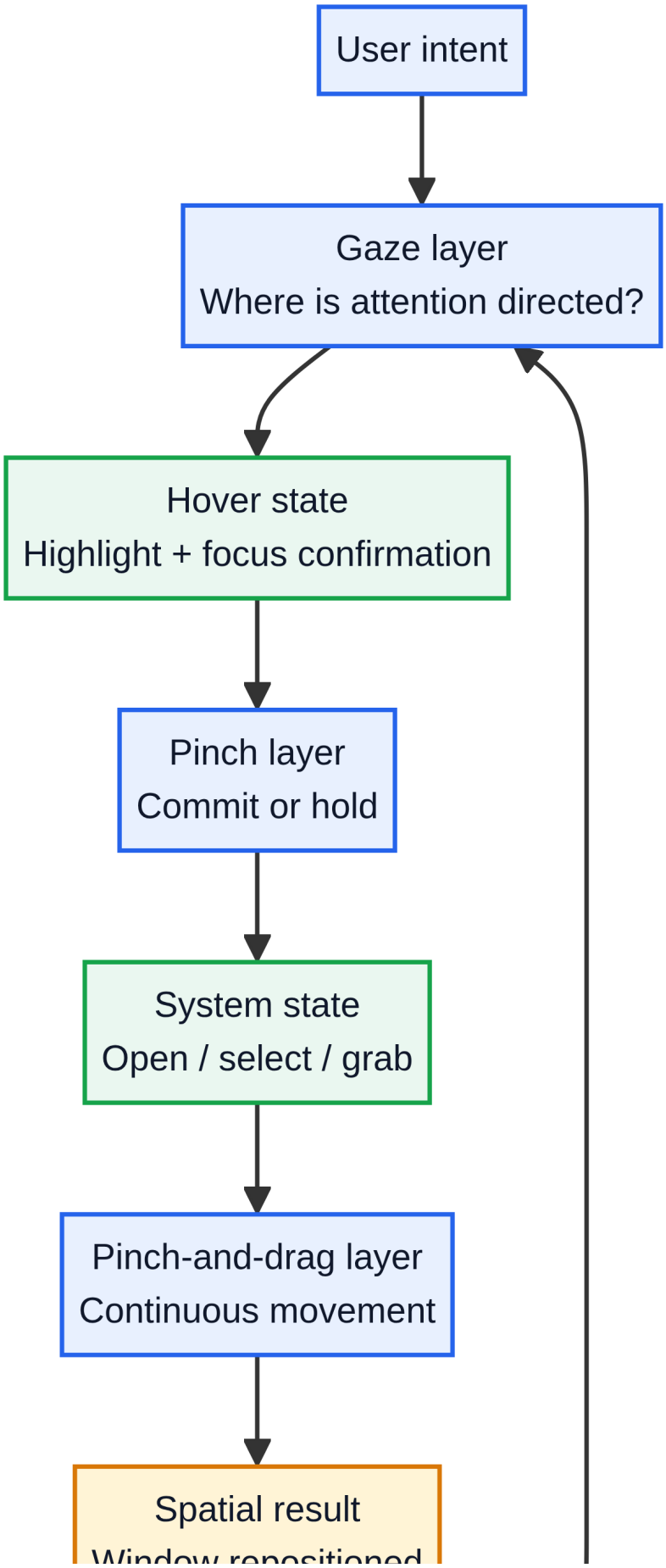

User intent
Gaze layer
Where is attention directed?
Hover state
Highlight + focus confirmation
Pinch layer
Commit or hold
System state
Open / select / grab
Pinch-and-drag layer
Continuous movement
Spatial result
Window repositioned

Figure 1. Proposed OmniSpace interaction architecture. The feedback loop keeps the user informed about the current target and state.

State logic for the proposed system

- Idle: no eligible target is focused.
- Hover: gaze identifies a target and the system shows focus feedback.
- Committed: a valid pinch activates the focused control or opens the selected option.
- Grabbed: a pinch-and-hold begins continuous manipulation of a draggable object.
- Released: the object is placed, the system confirms the result, and the state returns to idle or hover.

8. Gesture Interaction Flow Diagram

The following sequence illustrates the assigned Week 2 interaction path from menu targeting to window repositioning.



Figure 2. End-to-end gesture interaction flow for OmniSpace.

9. Error Handling and User Feedback

Spatial input is probabilistic. The system may not see a hand, may interpret a gaze ray differently from the user's intention, or may lose tracking during movement. Error handling should preserve the user's mental model: the interface should say what happened, avoid silent actions, and make the next attempt obvious.

Risk	Preventive behavior	Recovery feedback
Accidental selection	Require focused target + intentional pinch; never activate on gaze alone.	Keep target highlighted and show a short "Select with pinch" hint only when needed.
Missed gesture	Use generous hit areas and tolerate small tracking variation.	Show a subtle recognition cue; preserve focus for another attempt.
Hand-tracking loss	Check visibility and confidence before starting a grab.	Pause or release safely; prompt "Move hand into view" without moving the window unexpectedly.
User confusion	Keep state changes visually distinct and consistent.	Use clear labels, pressed/grabbed states, and reversible actions.
Unreachable window	Apply placement bounds and maintain a recenter or reset action.	Show a boundary cue and offer a quick "Return to workspace" action.

10. UX Recommendations

- Keep the gesture vocabulary small and consistent across menus, windows, and content.
- Use gaze for targeting and pinch for commitment so users do not fear that looking alone will trigger an action.
- Make hover feedback subtle, but make committed and grabbed states stronger.
- Prefer window bars or explicit handles for dragging; avoid making every surface draggable.
- Support comfortable hand positions. Apple advises that users do not need to hold their arms up and should keep hands visible for tracking.[1]
- Design for lighting, sleeves, gloves, jewelry, and occlusion as real-world conditions that can affect tracking.[1]
- Use head-gaze or another input fallback when eye-gaze is unavailable or when targets are too small. Microsoft identifies head-gaze as a useful fallback and notes that eye-gaze can be difficult for tiny targets.[3]
- Treat all timing thresholds, hit-area sizes, and motion offsets as hypotheses to validate in future usability testing.

11. Conclusion

Gesture mapping gives OmniSpace a clear interaction grammar for spatial work. Gaze Hover identifies what the user is attending to without committing an action. Pinch provides a discrete and familiar way to select or open the focused item. Pinch-and-Drag extends that pattern into continuous spatial manipulation for moving a floating window or object.

The most important architectural decision is the separation between targeting, commitment, and manipulation. This keeps the system predictable: looking highlights, pinching selects, and holding while moving repositions. It also creates a clear feedback contract. The user should always be able to tell what is focused, what has been selected, what is being held, and what happened after release.

This Week 2 report is a research-informed design specification, not a claim of implementation or user-test performance. The next responsible step would be to prototype the three states with representative spatial targets and evaluate recognition, accidental activation, comfort, recovery from tracking loss, and ease of learning with real users.

> **Final takeaway**
> A successful OmniSpace gesture system should feel calm and predictable: the user looks to target, pinches to commit, and pinches-and-drags only when the interface clearly shows that an object is being held.

Submission scope check

- Included: Week 2, Column 1 — UX Research & Spatial Architecture.
- Included: Gaze Hover, Pinch, Pinch-and-Drag, architecture, flow diagram, mapping table, feedback, and errors.
- Excluded: Figma component design, Spline material design, implementation claims, and invented testing results.

References

The following sources were accessed for this report. Platform documentation is used for established interaction guidance. The ACM paper is used to support the broader gaze-plus-pinch interaction concept. OmniSpace-specific timing, styling, and recovery recommendations remain design hypotheses for future validation.

[1] Apple Support. “Use gestures with Apple Vision Pro.” Official support documentation. https://support.apple.com/en-us/117741

[2] Apple. “Learn basic gestures and controls on Apple Vision Pro.” Apple User Guide. https://support.apple.com/guide/apple-vision-pro/basic-gestures-and-controls-tan1e2a29e00/visionos

[3] Microsoft Learn. “Gaze and commit.” Windows Mixed Reality design guidance. https://learn.microsoft.com/en-us/windows/mixed-reality/design/gaze-and-commit

[4] Pfeuffer, Ken, Benedikt Mayer, Diako Mardanbegi, and Hans Gellersen. “Gaze + pinch interaction in virtual reality.” Proceedings of the 5th Symposium on Spatial User Interaction, 2017, pp. 99–108. ACM. https://doi.org/10.1145/3131277.3132180

Citation notes

Claims about Apple Vision Pro gestures, hand visibility, comfortable hand position, and the look–pinch–drag pattern are grounded in Apple’s official documentation [1] [2]. Claims about gaze as a targeting signal, cursor or subtle feedback, head-gaze fallback, and gaze-plus-commit are grounded in Microsoft’s mixed-reality guidance [3]. The research framing for combining gaze with indirect freehand manipulation is grounded in the ACM paper by Pfeuffer and colleagues [4].

Report status

Prepared as a Week 2 internship submission report for OmniSpace – Spatial UI System. No mentor name or date has been invented. No experimental results or implementation status has been claimed.